On this page

Topology-Aware Resource Scheduling

Topology-Aware Resource Scheduling, or TARS for short, is the ability of distributing applications on a per node pool basis where a node pool is a set of Pulse instances.

This approach improves resiliency and resource management by:

- Isolating application processes, meaning that an application will keep running even if different node pools are degraded
- Giving the possibility of scaling machines according to the workload of each application

In this section you can find how TARS can be configured.

Overviewâ

As explained above TARS gives the possibility of distributing an application/set of applications in different node pools. The Pulse Server (PS) cluster will have to context of all deployed applications and acts as a single point of configuration for all.

In the following illustration you can find that:

- There are two node pools, `outbound` and `inbound`
- There are three Pulse instances, where two belong to `outbound` and one to `inbound`
- The Outbound application (Transfers and Digital Activity) is configured to be deployed in the `outbound` node pool
- The Inbound application is configured to be deployed in the `inbound` node pool
- The UI Load Balancer will connect to the Pulse leader node where the whole runtime cluster can be managed including the applications that are not deployed in the leader node
- The API Load Balancer on the other hand will only redirect traffic to the nodes that serve a specific application

This however does not mean one node pool can only serve one application. If, for instance, there are two applications that have a small workload they can be deployed in the same node pool.

The example below illustrates this case:

- The `Outbound` is still deployed in two (unshared) nodes
- The `Cards` application was configured to be deployed in the `inbound` node pool, hence sharing the node with the `Inbound` application
- The API Load Balancer for `Inbound` will now also redirect `Cards` requests for the `Cards` application running in the `inbound` node pool

Configurationâ

TARS configuration is controlled by the following Ansible variables:

- `apps_enabled` : should have all the applications to deploy
- `tars_enabled` : `true` to enable TARS, `false` otherwise
- `node_app_pool` : pool where the node belongs to, read from an environment variable
- `pulse_app_topology` : application topology in the structure defined below

```
pulse_app_topology:
  <app_1_id_as_defined_in_apps_enabled>:
    # should be the amount of instances across all pools (for apps without partitioned workflows)
    replication_factor: <app_1_replication_factor>
    app_pools:
```

```
      # dictionary of pools with the number of instances per pool
      <app_1_pool_name>:
        number_of_instances: <app_1_number_of_instances_in_pool>
  <app_2_id_as_defined_in_apps_enabled>:
    # should be the amount of instances across all pools (for apps without partitioned
workflows)
    replication_factor: <app_2_replication_factor>
    app_pools:
      # dictionary of pools with the number of instances per pool
      <app_2_pool_1_name>:
        number_of_instances: <app_2_number_of_instances_in_pool_1>
      <app_2_pool_2_name>:
        number_of_instances: <app_2_number_of_instances_in_pool_2>
```

Copy

The first thing to do regarding configuration is to define the topology you want to achieve. For the sake of this guide we will assume the topology defined in the second example given above, where `Inbound` and `Cards` will share a node pool and `Outbound` will have its own pool in two different nodes.

This means that when creating the instance groups they will need to have an environment variable indicating the pool they belong to. For this example the instance group for `Outbound` should have an environment variable `NODE_APP_POOL=outbound` and the `Inbound` / `Cards` should have its own set to `NODE_APP_POOL=inbound`.

warning

Since `node_app_pool` reads the node environment variable during deployment, the environment variable of each node needs to be set before running the deployment otherwise the pool will be configured as an empty string.

At this point with all the instance groups created we can continue configuration:

- Define the `apps_enabled` variable to include the applications to deploy

```
apps_enabled:
  - uk_outbound_payments
  - uk_reference_data
  - uk_inbound_payments
  - uk_cards
```

Copy

- Enable TARS by setting `tars_enabled: true`
- Define `pulse_app_topology` as follows:

```
pulse_app_topology:
  uk_outbound_payments:
    replication_factor: 2
    app_pools:
      outbound:
        number_of_instances: 2
  uk_reference_data:
    replication_factor: 1
    app_pools:
      outbound:
        number_of_instances: 1
  uk_inbound_payments:
    replication_factor: 1
    app_pools:
      inbound:
        number_of_instances: 1
  uk_cards:
    replication_factor: 1
    app_pools:
      inbound:
        number_of_instances: 1
```

Copy

Some notes regarding the `pulse_app_topology`:

- `uk_outbound_payments` has a `replication_factor` and `number_of_instances` of `2` because we want to have two instances of the `Outbound` application running in two different `outbound` nodes
- `uk_reference_data` will only be deployed in one of the `outbound` nodes
- `uk_inbound_payments` and `uk_cards` will be deployed in one `inbound` node

warning

Please note this example assumes the default `pulse_cluster_datacenters` variable was not overridden. In case it was the override needs to be extended as follows:

```
pulse_cluster_datacenters:
  - name:                              "dc1"
    numberofnodes:                     "{{ pulse_cluster_number_of_nodes }}"
    replicationfactor:                 "{{ pulse_cluster_replication_factor }}"
    (...)
    app_topology:                      "{{ pulse_app_topology if tars_enabled else [] }}"
```

Copy

After deployment you should be able to see how the applications were distributed in the Pulse UI by accessing the datacenter page in the administration tab.

Additional configurationâ□□

As stated before using TARS enables application distribution across different Pulse nodes while keeping the application configuration accessible via a singe Pulse UI.

This means that after a TARS deployment, the UI Load Balancer will need to be updated to include all the backends (Pulse instances) while choosing to where redirect traffic based on the leader node.

As for the API Load Balancer, given the example on this page, it will need to be configured as follows:

- Redirect `Transfers` and `Digital Activity` traffic to the `outbound` nodes
- Redirect `Inbound` and `Cards` traffic to the `inbound` nodes

info

Find a detailed description of all available configurations in Pulse Configuration Recommendations